

JAL2301 DATA SCIENCE

UNIT IV PYTHON LIBRARIES FOR DATA WRANGLING

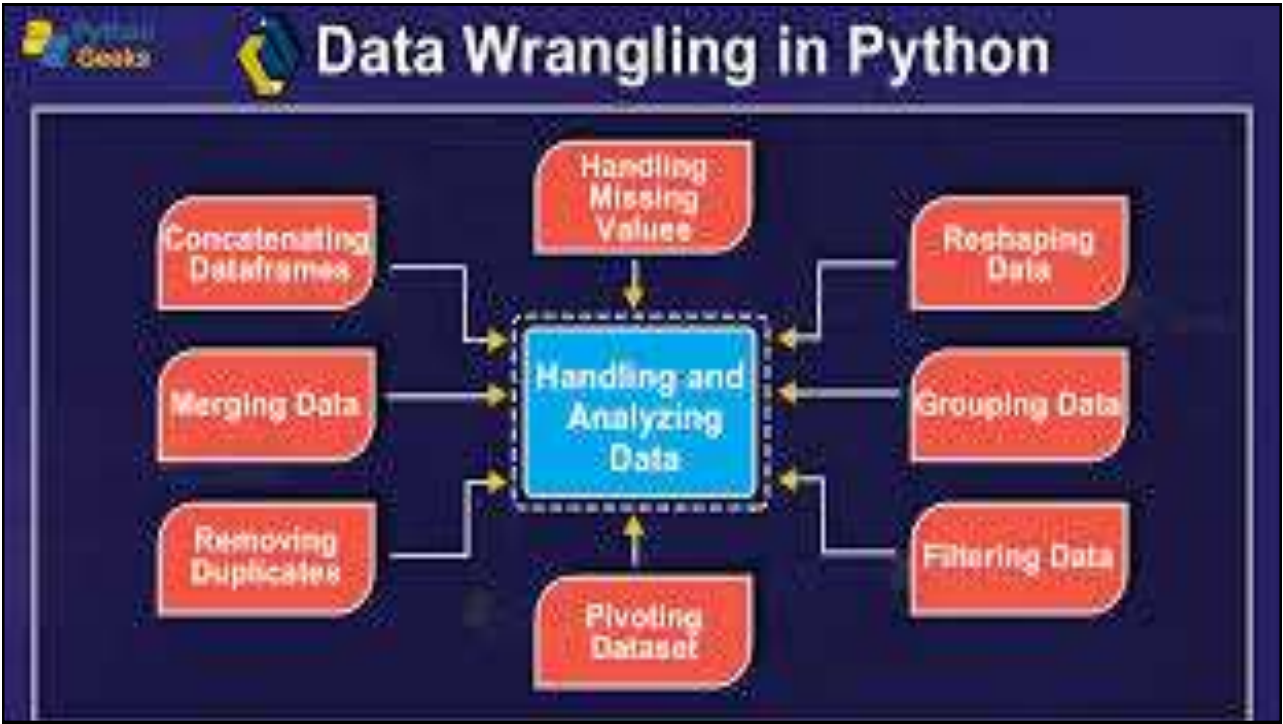


UNIT IV PYTHON LIBRARIES FOR DATA WRANGLING SYLLABUS

Basics of Numpy arrays – Aggregations
– Computations on arrays –
Comparisons masks boolean logic –
Fancy indexing – Sorting arrays –
Structured data

Data manipulation with Pandas – Data
indexing and selection – Operating on
data – Handling missing data –
Hierarchical indexing – Combining
datasets – Pivot tables.









**DATA
MANIPULATION
WITH PANDAS**

PANDAS

- NumPy and its ndarray object provides efficient storage and manipulation of dense typed arrays in Python.
- NumPy's ndarray data structure provides essential features for the type of clean, well-organized data typically seen in numerical computing tasks.

Limitations:

- → less flexibility (attaching labels to data, working with missing data, etc.) and
- → when attempting operations that do not map well to element-wise broadcasting (groupings, pivots, etc.),
- Above mentioned are important piece of analyzing the less structured data available in many forms in the world around us.

5

PANDAS

- Pandas library provides efficient data structures.
- Pandas → a newer package built on top of NumPy → provides an efficient implementation of a DataFrame.
- DataFrames → essentially multidimensional arrays with attached row and column labels, and often with heterogeneous types and/or missing data.
- Pandas and its Series and DataFrame objects, builds on the NumPy array structure
- Provides efficient access to these sorts of data munging (data wrangling) tasks that occupy much of a data scientist's time.

6



DATA INDEXING AND SELECTION

DATA INDEXING AND SELECTION

- **DATA SELECTION IN SERIES**

- SERIES AS DICTIONARY
- SERIES AS 1-D ARRAY
- INDEXERS: LOC, ILOC, AND IX

- **DATA SELECTION IN DATAFRAME**

- DATAFRAME AS A DICTIONARY
- DATAFRAME AS 2 – D ARRAY
- ADDITIONAL INDEXING CONVENTIONS



DATA INDEXING AND SELECTION

- Methods and tools to access, set, and modify values in NumPy arrays
→ indexing, slicing, masking, fancy indexing and combinations.
- Similar means of accessing and modifying values in Pandas Series and DataFrame objects are available.
- Simpler case → Data selection in 1-d series object
- Complicated case → Data selection in 2-d data frame object



9

DATA SELECTION IN SERIES - SERIES AS DICTIONARY

Series object

- A **Pandas Series** is like a column in a table.
- It is a one-dimensional array holding data of any type.
- Provides a mapping from a collection of keys to a collection of values.

Output:

```
a  0.25
b  0.50
c  0.75
d  1.00
dtype: float64
```

```
import pandas as pd
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])
print(data)
```

```
print(data['b'])
```

0.5

10

DATA SELECTION IN SERIES - SERIES AS DICTIONARY

Series object

- Dictionary-like Python expressions and methods are available to examine the keys/indices and values
- A Series can be extended by assigning to a new index value:

```
print('b' in data)
print(data.keys())
print(list(data.items()))
data['e'] = 1.62
```

Output:

```
True
Index(['a', 'b', 'c', 'd'], dtype='object')
[('a', 0.25), ('b', 0.5), ('c', 0.75), ('d', 1.0)]
```

- Pandas is making decisions about memory layout and data copying

DATA SELECTION IN SERIES - SERIES AS 1-DIMENSIONAL ARRAY

Series object

- Can also be used for slicing, masking and fancy indexing

```
print(data['a' : 'c'])           # slicing by explicit integer index
print(data[0 : 2])              # slicing by implicit integer index
print(data[(data > 0.3) & (data < 0.8)])  # masking
print(data[['a', 'e']])        # fancy indexing
```

Output 1:

```
a    0.25
b    0.50
c    0.75
dtype: float64
```

Output 2:

```
a    0.25
b    0.50
dtype: float64
```

Output 3:

```
b    0.50
c    0.75
dtype: float64
```

Output 4:

```
a    0.25
e    1.62
dtype: float64
```

DATA SELECTION IN SERIES - INDEXERS: LOC, ILOC, AND IX

Series object

- If Series has integer index, it creates confusion in indexing and slicing.

```
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=[1, 2, 3, 4])
print(data[1])           # explicit index when indexing
print(data[0 : 2])       # implicit index when slicing
```

Output 1:
0.25

Output 2:
1 0.25
2 0.50
dtype: float64



DATA SELECTION IN SERIES - INDEXERS: LOC, ILOC, AND IX

Series object

- To avoid confusion, loc (for explicit indexing) and iloc (for implicit indexing) can be used. (ix indexer – used in the context of DataFrame objects)

```
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=[1, 2, 3, 4])
print(data.loc[1])       # explicit index when indexing
print(data.loc[1 : 3])   # explicit index when slicing
print(data.iloc[1])      # implicit index when indexing
print(data.iloc[1 : 3])  # implicit index when slicing
```

Output 1:
0.25

Output 2:
1 0.25
2 0.50
3 0.75
dtype: float64

Output 3:
0.50

Output 4:
2 0.50
3 0.75
dtype: float64

DATA SELECTION IN DATAFRAME - DATAFRAME AS A DICTIONARY

DataFrame object

- DataFrame acts like a two-dimensional or structured array → like a **dictionary of Series structures** sharing the same index.

```
import pandas as pd
area = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])
pop = pd.Series([100, 150, 125, 85], index=['a', 'b', 'c', 'd'])
print(data) = pd.DataFrame({'AREA' : area, 'POPULATION' : pop})
```

Output:		
	AREA	POPULATION
a	0.25	100
b	0.50	150
c	0.75	125
d	1.00	85

15

DATA SELECTION IN DATAFRAME - DATAFRAME AS A DICTIONARY

DataFrame object

- DataFrame acts like a two-dimensional or structured array → like a **dictionary of Series structures** sharing the same index.

```
import pandas as pd
area = pd.Series({'a' : 0.25, 'b' : 0.5, 'c' : 0.75, 'd' : 1.0})
pop = pd.Series({'a' : 100, 'b' : 150, 'c' : 125, 'd' : 85})
data = pd.DataFrame({'AREA' : area, 'POPULATION' : pop})
```

Output:		
	AREA	POPULATION
a	0.25	100
b	0.50	150
c	0.75	125
d	1.00	85

16

DATA SELECTION IN DATAFRAME - DATAFRAME AS A DICTIONARY

DataFrame object

- Accessing the individual Series

```
print(data['AREA'])          (or)          print(data.AREA)
```

Output for both cases:

```
a  0.25
b  0.50
c  0.75
d  1.00
Name: AREA, dtype: float64
```

17

DATA SELECTION IN DATAFRAME - DATAFRAME AS A DICTIONARY

DataFrame object

- Modifying the object → eg. adding a new column

```
data['DENSITY'] = data['POPULATION'] / data['AREA']
print(data)
```

Output for both cases:

	AREA	POPULATION	DENSITY
a	0.25	100	400.000000
b	0.50	150	300.000000
c	0.75	125	166.666667
d	1.00	85	85.000000

18

DATA SELECTION IN DATAFRAME - DATAFRAME AS A 2-D ARRAY

DataFrame object

- Full DataFrame **transposed** to swap rows and columns:

```
print(data)
print(data.T)
```

Output 1:

	AREA	POPULATION	DENSITY
a	0.25	100	400.000000
b	0.50	150	300.000000
c	0.75	125	166.666667
d	1.00	85	85.000000

Output 2:

	a	b	c	d
AREA	0.25	0.5	0.750000	1.0
POPULATION	100.00	150.0	125.000000	85.0
DENSITY	400.00	300.0	166.666667	85.0

DATA SELECTION IN DATAFRAME - DATAFRAME AS A 2-D ARRAY

DataFrame object

- Raw underlying data array can be accessed using the values attribute

```
print(data.values)
print(data.values[0]) # similar to array indexing
```

```
[[2.50000000e-01 1.00000000e+02 4.00000000e+02]
 [5.00000000e-01 1.50000000e+02 3.00000000e+02]
 [7.50000000e-01 1.25000000e+02 1.66666667e+02]
 [1.00000000e+00 8.50000000e+01 8.50000000e+01]]
```

DATA SELECTION IN DATAFRAME - DATAFRAME AS A 2-D ARRAY

DataFrame object

```
print(data['AREA'])    #a single "index" passed to a DataFrame to
                        #access a column
print(data.iloc[:3, :2])    # array slicing – using implicit indexing
print(data.loc['a' : 'c'])  # array slicing – using explicit indexing
```

Output 1:

a 0.25
b 0.50
c 0.75
d 1.00
Name: AREA, dtype: float64

Output 2:

	AREA	POPULATION
a	0.25	100
b	0.50	150
c	0.75	125

Output 3:

	AREA	POPULATION	DENSITY
a	0.25	100	400.000000
b	0.50	150	300.000000
c	0.75	125	166.666667

DATA SELECTION IN DATAFRAME - DATAFRAME AS A 2-D ARRAY

DataFrame object

```
print(data.ix[:3, : 'POPULATION'])    # array slicing – using HYBRID indexing
```

Rows 0 to 2, columns 'AREA' TO 'POPULATION'

Output:

	AREA	POPULATION
a	0.25	100
b	0.50	150
c	0.75	125

DATA SELECTION IN DATAFRAME - DATAFRAME AS A 2-D ARRAY

DataFrame object

- If you need the indices with population > 100.

```
print(data.loc[data.POPULATION >= 100])    # array masking
print(data.loc[data.POPULATION >= 100, ['AREA', 'POPULATION']])
```

Output 1:

	AREA	POPULATION	DENSITY
a	0.25	100	400.000000
b	0.50	150	300.000000
c	0.75	125	166.666667

Output 2:

	AREA	POPULATION
a	0.25	100
b	0.50	150
c	0.75	125





OPERATING ON DATA

OPERATING ON DATA

- **UFUNCS: INDEX PRESERVATION**
- **UFUNCS: INDEX ALIGNMENT**
 - INDEX ALIGNMENT IN SERIES
 - INDEX ALIGNMENT IN DATAFRAME



25

OPERATING ON DATA

- Pandas inherits much of ufuncs from NumPy perform quick element-wise to perform operations, both with basic arithmetic (addition, subtraction, multiplication, etc.) and with more sophisticated operations (trigonometric functions, exponential and logarithmic functions, etc.).
- Pandas will preserve index and column labels in the output.
- Pandas will automatically align indices when passing the objects to the ufunc.

26

OPERATING ON DATA - UFUNCS: INDEX PRESERVATION

- Consider an example Pandas Series and DataFrame objects.

```
import pandas as pd
import numpy as np
range1 = np.random.RandomState(42)
series1 = pd.Series(range1.randint(0, 10, 4))
series2 = pd.Series(np.random.randint(0, 10, 4))
print(series1)    # same set of values for every execution
print(series2)    # different set of values for each execution
```

Output 1:

0	6
1	3
2	7
3	4
dtype: int32	

Output 2:

0	3
1	2
2	9
3	3
dtype: int32	

OPERATING ON DATA - UFUNCS: INDEX PRESERVATION

- Consider an example Pandas Series and DataFrame objects.

```
import pandas as pd
import numpy as np
range1 = np.random.RandomState(42)
df1 = pd.DataFrame(range1.randint(0, 10, (3, 4)), columns=['A', 'B', 'C', 'D'])
```

Output:

	A	B	C	D
0	6	9	2	6
1	7	4	3	7
2	7	2	5	4

OPERATING ON DATA - UFUNCS: INDEX PRESERVATION

- Ufunc applied on those objects - the result will be another Pandas object with the **indices preserved** – **indices will appear in the o/p.**

```
print(np.exp(series1) # np.log, np.log, np.log10 can also be used
print(np.sin(df1))
```

Output 1:

0	403.428793
1	20.085537
2	1096.633158
3	54.598150

dtype: float64

Output 2:

	A	B	C	D
0	-0.279415	0.412118	0.909297	-0.279415
1	0.656987	-0.756802	0.141120	0.656987
2	0.656987	0.909297	-0.958924	-0.756802

OPERATING ON DATA - UFUNCS: INDEX ALIGNMENT – IN SERIES

- For binary operations on two Series or DataFrame objects, Pandas will align indices in the process of performing the operation

```
area = pd.Series({'a' : 0.25, 'b' : 0.5, 'c' : 0.75, 'd' : 1.0}, name = 'AREA')
pop = pd.Series({'a' : 100, 'b' : 150, 'c' : 125, 'd' : 85}, name = 'POPULATION')
print(pop / area)
area1 = pd.Series({'a' : 0.25, 'b' : 0.5, 'd' : 1.0}, name = 'AREA')
pop1 = pd.Series({'b' : 150, 'c' : 125, 'a' : 100}, name = 'POP')
print(pop1 / area1)
```

Output 1:

a	400.000000
b	300.000000
c	166.666667
d	85.000000

dtype: float64

Output 2:

a	400.0
b	300.0
c	NaN
d	NaN

dtype: float64

- Result has the union of all indices
- NaN → Not a Number → missing data

OPERATING ON DATA - UFUNCS: INDEX ALIGNMENT – IN SERIES

- For binary operations on two Series or DataFrame objects, Pandas will align indices in the process of performing the operation

```
A = pd.Series([2, 4, 6, 5], index=[1, 2, 3, 4])
B = pd.Series([1, 3, 5, 7], index=[1, 2, 3, 4])
print(A + B)

A = pd.Series([2, 4, 6], index=[0, 1, 2])
B = pd.Series([1, 3, 5], index=[1, 2, 3])
print(A + B)
```

Output 1:

```
1    3
2    7
3   11
4   12
dtype: int64
```

Output 2:

```
0    NaN
1    5.0
2    9.0
3    NaN
dtype: float64
```

- Result has the union of all indices
- NaN → Not a Number → missing data

OPERATING ON DATA - UFUNCS: INDEX ALIGNMENT – IN SERIES

- For binary operations on two Series or DataFrame objects, Pandas will align indices in the process of performing the operation

```
A = pd.Series([2, 4, 6], index=[0, 1, 2])
B = pd.Series([1, 3, 5], index=[1, 2, 3])
print(A.add(B, fill_value=0)) # explicit specification of the fill value for any
elements in A or B that might be missing
```

Output 1:

```
0    2.0
1    5.0
2    9.0
3    5.0
dtype: float64
```

OPERATING ON DATA - UFUNCS: INDEX ALIGNMENT – IN DATAFRAME

- For binary operations on two Series or DataFrame objects, Pandas will align indices in the process of performing the operation

```
range1 = np.random.RandomState(42)
df1 = pd.DataFrame(range1.randint(0, 20, (2, 2)), columns=['A', 'B'])
df2 = pd.DataFrame(range1.randint(0, 10, (3, 3)), columns=list('BAC'))
print(df1, df2)
print(df1 + df2)
```

OPERATING ON DATA - UFUNCS: INDEX ALIGNMENT – IN DATAFRAME

- Fill the missing values with the mean of all values in A

```
range1 = np.random.RandomState(42)
df1 = pd.DataFrame(range1.randint(0, 20, (2, 2)), columns=['A', 'B'])
df2 = pd.DataFrame(range1.randint(0, 10, (3, 3)), columns=list('BAC'))
fill = df1.stack().mean() # fill = 12.25 for the output
print(df1.add(df2, fill_value=fill))
```

<u>Output 1:</u>		
	A	B
0	6	19
1	14	10

<u>Output :</u>			
	B	A	C
0	7	4	6
1	9	2	6
2	7	4	3

<u>Output 2:</u>			
	A	B	C
0	10.00	26.00	18.25
1	16.00	19.00	18.25
2	16.25	19.25	15.25



HANDLING MISSING DATA

- **TRADE-OFFS IN MISSING DATA CONVENTIONS**
- **MISSING DATA IN PANDAS**
 - **NONE: PYTHONIC MISSING DATA**
 - **NAN: MISSING NUMERICAL DATA**
 - **NAN AND NONE IN PANDAS**
- **OPERATING ON NULL VALUES**
 - **DETECTING NULL VALUES**
 - **DROPPING NULL VALUES**
 - **FILLING NULL VALUES**

HANDLING MISSING DATA

- Real-world data is rarely clean and homogeneous.
- Will have some amount of data missing
- Different data sources may indicate missing data in different ways
- Pandas chooses to represent it as null, NaN, or NA values
- built-in Pandas tools available for handling missing data in Python



37

TRADE-OFFS IN MISSING DATA CONVENTIONS

- Two strategies to indicate the presence of missing data in a table or DataFrame.:
 1. **Masking approach** - Using a mask that globally indicates missing values – a Boolean array or one bit in the data representation to locally indicate the null status of a value
 2. **Sentinel approach** - Choosing a sentinel value that indicates a missing entry - some data-specific convention, such as indicating a missing integer value with –9999 or some rare bit pattern, or it could be a more global convention, such as indicating a missing floating-point value with NaN (Not a Number), a special value.

38

TRADE-OFFS IN MISSING DATA CONVENTIONS

- Trade-offs in the two strategies:
 - **Masking approach** - use of a separate mask array requires allocation of an additional Boolean array → adds overhead in both storage and computation.
 - **Sentinel approach** - A sentinel value reduces the range of valid values and special values like NaN are not available for all data types.

39

MISSING DATA IN PANDAS - NONE: PYTHONIC MISSING DATA

- None → a Python singleton **object** often used for missing data in Python code → can be used only in arrays with data type 'object' (i.e., arrays of Python objects)

```
import numpy as np
import pandas as pd
vals1 = np.array([1, None, 3, 4])
print(vals1)
print(sum(vals1))
```

Output:

```
array([1, None, 3, 4], dtype=object)
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

MISSING DATA IN PANDAS - NAN: MISSING NUMERICAL DATA

- NAN → acronym for Not a Number → a special **floating-point** value recognized by all systems that use the standard floating point representation.
- NaN is a bit like a data virus → it infects any other object it touches → this means that aggregates over the values are well defined (i.e., they don't result in an error) but result in nan → some special aggregations that will ignore these missing values

```
import numpy as np
import pandas as pd
vals2 = np.array([1, np.nan, 3, 4])
print(vals2)
print(vals2.dtype)
print(sum(vals2))
print(1 + np.nan)
print(np.nansum(vals2))
```

Output:

[1. nan 3. 4.]

float64

nan

Nan

8.0

MISSING DATA IN PANDAS - NAN AND NONE IN PANDAS

- Pandas is built to handle NAN and NONE nearly interchangeably, converting where appropriate
- For example, if we set a value in an integer array to np.nan, it will automatically be upcast to a floating-point type to accommodate the NA
- Sometimes, Pandas automatically converts the None to a NaN value.

```
print(pd.Series([1, np.nan, 2, None]))

x = pd.Series(range(2), dtype=int)
print(x)

x[0] = None
print(x)
```

Output1:

0 1.0

1 NaN

2 2.0

3 NaN

dtype:

float64

Output2:

0 0

1 1

dtype:

int64

Output3:

0 NaN

1 1.0

dtype:

float64

OPERATING ON NULL VALUES

- Pandas methods for detecting, removing, and replacing null values in Pandas data structures.
- `isnull()` → Generate a Boolean mask indicating missing values
- `notnull()` → Opposite of `isnull()`
- `dropna()` → Return a filtered version of the data

Detecting null values

```
data =pd.Series([10, np.nan, 20, None])
print(data.isnull())
print(data.notnull())
```

Dropping null values in a series

```
print(data.dropna())
```

Output1:

```
0 False
1 True
2 False
3 True
dtype: bool
```

Output2:

```
0 10
2 20
dtype:
object
```

Output3:

```
0 10
2 20
dtype:
object
```

OPERATING ON NULL VALUES

- Pandas methods for detecting, removing, and replacing null values in Pandas data structures.
- `isnull()` → Generate a Boolean mask indicating missing values
- `notnull()` → Opposite of `isnull()`
- `dropna()` → Return a filtered version of the data

Dropping null values in a dataframe

```
df = pd.DataFrame([[1, np.nan, 2], [2, 3, 5], [np.nan, 4, 6]])
print(df.dropna()) # entire row with nan dropped
```

```
print(df.dropna(axis = 1)) # entire column with nan dropped
df.dropna(axis='columns', how='all') #drops column if all are NAN
df.dropna(axis='columns', how='any') #drops column if any is NAN
```

Output:

```

0 1 2
1 2.0 3.0 5
2
0 2
1 5
2 6
```

OPERATING ON NULL VALUES

• fillna() → Return a copy of the data with missing values filled

```
# Filling null values in a series
data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))
print(data.fillna(0))
print(data.fillna(method='ffill')) # forward-fill
print(data.fillna(method='bfill')) # backward-fill
```

Output 1:
a 1.0
b 0.0
c 2.0
d 0.0
e 3.0
dtype: float64

Output 2:
a 1.0
b 1.0
c 2.0
d 2.0
e 3.0
dtype: float64

Output 1:
a 1.0
b 2.0
c 2.0
d 3.0
e 3.0
dtype: float64



COMBINING DATASETS

COMBINING DATASETS

- **CONCAT AND APPEND**

- NP.CONCATENATE FUNCTION
- PD.CONCAT METHOD
- PD.CONCAT WITH INNER JOIN
- APPEND METHOD

- **MERGE AND JOIN**

- ONE-TO-ONE JOIN
- MANY-TO-ONE JOIN



49

COMBINING DATASETS

- Combining different data sources
- Simpler concatenation of two different datasets
- Complicated database-style joins and merges - handling any overlaps between the datasets
- Pandas includes functions and methods that make this sort of data wrangling fast and straightforward – on Series and DataFrames
- Function used - pd.concat

50

COMBINING DATASETS

- A DataFrame created

```
import numpy as np
import pandas as pd
cols = pd.Series(['A', 'B', 'C'])
ind = np.arange(3)
vals = {c: [str(c) + str(i) for i in ind] for c in cols}
data = pd.DataFrame(vals, ind)
print(data)
```

Output:

	A	B	C
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

51

COMBINING DATASETS

- Concatenation of NumPy arrays → done via the **np.concatenate** function

```
x = [1, 2, 3]
y = [4, 5, 6]
data = np.concatenate([x, y])
print(data)
x = [[1, 2], [3, 4]]
data1 = np.concatenate([x, x], axis=0)
data2 = np.concatenate([x, x], axis=1)
print(data1, data2)
```

Output:

[1, 2, 3, 4, 5, 6]

[[1 2 1 2]
[3 4 3 4]]

[[1 2]
[3 4]
[1 2]
[3 4]]

COMBINING DATASETS – CONCATENATE & APPEND

- Concatenation of Series and DataFrame objects is very similar to concatenation of NumPy arrays →
 - Done via the **pd.concat** function
 - Done via **append** method

```
pd.concat(objs, axis=0, join='outer', join_axes=None, ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, copy=True)
```

COMBINING DATASETS – CONCATENATE & APPEND

- To use **pd.concat** function, for row-wise concatenation, column names must be the same in both dataframes

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']},index=[1,2])
df2 = pd.DataFrame({'COL1':['A3', 'A4'],'COL2':['B3', 'B4']}, index=[3,4])
print(df1)
print(df2)
print(pd.concat([df1, df2]))           # row-wise concatenation
```

Output 1:

	COL1	COL2
1	A1	B1
2	A2	B2

Output 2:

	COL1	COL2
3	A3	B3
4	A4	B4

Output 3:

	COL1	COL2
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4

COMBINING DATASETS – CONCATENATE & APPEND

- To use **pd.concat** function, for column-wise concatenation (with axis=1), row indices names must be the same in both dataframes

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']}, index=[1,2])
df2 = pd.DataFrame({'COL3':['A3', 'A4'],'COL4':['B3', 'B4']}, index=[1,2])
print(df1)
print(df2)
print(pd.concat([df1, df2], axis=1))    # column-wise concatenation
```

Output 1:			Output 2:			Output 3:			
COL1 COL2			COL1 COL2			COL1 COL2 COL3 COL4			
1	A1	B1	3	A3	B3	1	A1	B1	A3 B3
2	A2	B2	4	A4	B4	2	A2	B2	A4 B4

COMBINING DATASETS – CONCATENATE & APPEND

- To use **pd.concat** function, for row-wise concatenation, if column names are not the same in both dataframes, union of col names done, and NaN used for missing values

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']},index=[1,2])
df2 = pd.DataFrame({'COL2':['A3', 'A4'],'COL3':['B3', 'B4']}, index=[3,4])
print(df1)
print(df2)
print(pd.concat([df1, df2]))    # row-wise concaten
```

Output 1:			Output 2:			Output 3:			
COL1 COL2			COL2 COL3			COL1 COL2 COL3			
1	A1	B1	3	A3	B3	1	A1	B1	NaN
2	A2	B2	4	A4	B4	2	A2	B2	NaN
						3	NaN	A3	B3
						4	NaN	A4	B4

COMBINING DATASETS – CONCATENATE & APPEND

- To avoid Nan, an intersection of the columns using join='inner' in pd.concat can be used

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']},index=[1,2])
df2 = pd.DataFrame({'COL2':['A3', 'A4'],'COL3':['B3', 'B4']}, index=[3,4])
print(df1)
print(df2)
print(pd.concat([df1, df2], join='inner'))    # row-wise inner join
```

Output 1:

	COL1	COL2
1	A1	B1
2	A2	B2

Output 2:

	COL2	COL3
3	A3	B3
4	A4	B4

Output 3:

	COL2
1	B1
2	B2
3	A3
4	A4

COMBINING DATASETS – CONCATENATE & APPEND

- To avoid Nan, an intersection of the columns using join='inner' in pd.concat can be used

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']},index=[1,2])
df2 = pd.DataFrame({'COL2':['A3', 'A4'],'COL3':['B3', 'B4']}, index=[3,4])
print(df1)
print(df2)
print(pd.concat([df1, df2], join='inner'))    # row-wise inner join
```

Output 1:

	COL1	COL2
1	A1	B1
2	A2	B2

Output 2:

	COL2	COL3
3	A3	B3
4	A4	B4

Output 3:

	COL2
1	B1
2	B2
3	A3
4	A4

COMBINING DATASETS – CONCATENATE & APPEND

- Concatenation using append method –returns a dataframe object – not so efficient as it creates a new index and data buffer
- Multiple append() == pd concat() → to concatenate more dataframes

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']},index=[1,2])
df2 = pd.DataFrame({'COL2':['A3', 'A4'],'COL3':['B3', 'B4']}, index=[3,4])
print(df1)
print(df2)
print(df1.append(df2))
```

Output 1:

	COL1	COL2
1	A1	B1
2	A2	B2

Output 2:

	COL1	COL2
3	A3	B3
4	A4	B4

Output 3:

	COL1	COL2
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4

COMBINING DATASETS – CONCATENATE & APPEND

- Concatenation using append method –returns a dataframe object – not so efficient as it creates a new index and data buffer
- Multiple append() == pd concat() → to concatenate more dataframes

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']},index=[1,2])
df2 = pd.DataFrame({'COL2':['A3', 'A4'],'COL3':['B3', 'B4']}, index=[3,4])
print(df1)
print(df2)
print(df1.append(df2))
```

Output 1:

	COL1	COL2
1	A1	B1
2	A2	B2

Output 2:

	COL1	COL2
3	A3	B3
4	A4	B4

Output 3:

	COL1	COL2
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4

COMBINING DATASETS – MERGE & JOIN

- `pd.merge()` → a subset of what is known as relational algebra,
- A formal set of rules for manipulating relational data, and forms the conceptual foundation of operations available in most databases
- Several primitive operations, which become the building blocks of more complicated operations on any dataset
- `pd.merge()` function and the related `join()` method of Series and DataFrames.
- The `pd.merge()` function implements a number of types of joins: the one-to-one, many-to-one, and many-to-many joins



61

COMBINING DATASETS – MERGE & JOIN

- One – to – one join

```
df1 = pd.DataFrame({'COL1':['A1', 'A2'],'COL2':['B1', 'B2']})
df2 = pd.DataFrame({'COL1':['A2', 'A1'],'COL3':['B3', 'B4']})
print(df1)
print(df2)
print(pd.merge(df1, df2))
```

Output 1:

	COL1	COL2
1	A1	B1
2	A2	B2

Output 2:

	COL1	COL3
0	A2	B3
1	A1	B4

Output 3:

	COL1	COL2	COL3
0	A1	B1	B4
1	A2	B2	B3

COMBINING DATASETS – MERGE & JOIN

- One – to – one join → similar to the column-wise concatenation seen
- No. of values in both dataframes to be merged are the same

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'],
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue'],
                    'hire_date': [2004, 2008, 2012, 2014]})
print(df1)
print(df2)
df3 = pd.merge(df1, df2)
print(df3)
```

Output 3:

	COL1	COL2	COL3
0	A1	B1	B4
1	A2	B2	B3

COMBINING DATASETS – MERGE & JOIN

- One – to – one join

df1			df2		
	employee	group		employee	hire_date
0	Bob	Accounting	0	Lisa	2004
1	Jake	Engineering	1	Bob	2008
2	Lisa	Engineering	2	Jake	2012
3	Sue	HR	3	Sue	2014



	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

COMBINING DATASETS – MERGE & JOIN

- Many – to – one join → one of the two key columns contains duplicate entries.

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'],
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR']})

df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue'],
                    'hire_date': [2004, 2008, 2012, 2014]})

df3 = pd.merge(df1, df2)
print(df3)

df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],
                    'supervisor': ['Carly', 'Guido', 'Steve']})
print(pd.merge(df3, df4))
```

COMBINING DATASETS – MERGE & JOIN

- Many – to – one join → one of the two key columns contains duplicate entries.

df3				df4		
	employee	group	hire_date		group	supervisor
0	Bob	Accounting	2008	0	Accounting	Carly
1	Jake	Engineering	2012	1	Engineering	Guido
2	Lisa	Engineering	2004	2	HR	Steve
3	Sue	HR	2014			

Df3, df4 merged					
	employee	group	hire_date	supervisor	
0	Bob	Accounting	2008	Carly	
1	Jake	Engineering	2012	Guido	
2	Lisa	Engineering	2004	Guido	
3	Sue	HR	2014	Steve	



PIVOT TABLES

PIVOT TABLES

- PIVOT TABLES
- MULTILEVEL PIVOT TABLES
- ADDITIONAL PIVOT TABLE OPTIONS

PIVOT TABLES

- The pivot table → takes simple columnwise data as input, and groups the entries into a two-dimensional table that provides a multidimensional summarization of the data.
- Pivot tables → multidimensional version of GroupBy aggregation → split and the combine happen across not a one-dimensional index, but across a two-dimensional grid.
- Method used → pivot_table()



69

PIVOT TABLES

- Sample database used → database of passengers on the Titanic

```
import numpy as np
import pandas as pd
import seaborn as sns
titanic = sns.load_dataset('titanic')
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

PIVOT TABLES

- Example Groupby operation → to find the survival rate by gender:
`titanic.groupby('sex')[['survived']].mean()`
- Finds different groups based on 'sex' – then finds the mean of 'survived' for different values in 'sex' (female, male) in the database

Output:

	survived
sex	
female	0.742038
male	0.188908

Three of every four ($3/4 = 0.75$) females on board survived, while only one in five males ($1/5 = 0.2$) survived

71

PIVOT TABLES

- Example Groupby operation → to find the survival rate by gender and class
→ group by class and gender, select survival, apply a mean aggregate, combine the resulting groups, and then unstack to reveal the hidden multidimensionality.

`titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack()`

Output:

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

Class-wise gender-wise survivals → multidimensional aggregation operation

72

PIVOT TABLES

- Multidimensional aggregation can be handled simply using 'pivot table'
→ method used is pivot_table()

`DataFrame.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean', fill_value=None, margins=False, dropna=True, margins_name='All', observed=False, sort=True)`

- data → the data (column) on which pivot table is to be generated
- index → row titles
- columns → column titles
- aggfunc → type of aggregation applied → 'mean' by default
- fill_value & dropna → to handle missing values
- margins → if True → totals computed along each grouping (row-wise and column-wise)
- observed → if True, shows observed values for categorical groupers
- sort → if True, results are sorted

73

PIVOT TABLES

- The multidimensional aggregation operation using groupby

`titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack()`

- The equivalent operation using pivot_table()

`titanic.pivot_table('survived', index='sex', columns='class')`

Output:

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

Class-wise gender-wise survivals →
2-dimensional operation

74

MULTILEVEL PIVOT TABLES

- The grouping in pivot tables can be specified with multiple levels, and via a number of options.
- Example – let age be the 3rd dimension (apart from class & sex)

```
age = pd.cut(titanic['age'], [0, 18, 80])
titanic.pivot_table('survived', ['sex', age], 'class')
```

- pd.cut() will split the age data into 2 sets / bins → from 0 to 18 (inclusive) and 19 to 80

Output:

		First	Second	Third
class				
sex	age			
female	(0, 18]	0.909091	1.000000	0.511628
	(18, 80]	0.972973	0.900000	0.423729
male	(0, 18]	0.800000	0.600000	0.215686
	(18, 80]	0.375000	0.071429	0.133663

Class-wise gender-wise age-wise survivals → 3-dimensional operation

MULTILEVEL PIVOT TABLES

- Example – let fare be the 4th dimension (apart from sex, age, class)

```
fare = pd.qcut(titanic['fare'], 2)
titanic.pivot_table('survived', ['sex', age], [fare, 'class'])
```

- pd.qcut() will split the 'fare' data into 2 sets / bins → based on quantile.
- Output: 4-dimensional aggregation

		[0, 14.454]		
class		First	Second	Third
sex	age			
female	(0, 18]	NaN	1.000000	0.714286
	(18, 80]	NaN	0.880000	0.444444
male	(0, 18]	NaN	0.000000	0.260870
	(18, 80]	0.0	0.098039	0.125000

		(14.454, 512.329]		
class		First	Second	Third
sex	age			
female	(0, 18]	0.909091	1.000000	0.318182
	(18, 80]	0.972973	0.914286	0.391304
male	(0, 18]	0.800000	0.818182	0.178571
	(18, 80]	0.391304	0.030303	0.192308

ADDITIONAL PIVOT TABLE OPTIONS

- Example: aggregation functions applied

```
titanic.pivot_table (index='sex', columns='class', aggfunc = {'survived': sum, 'fare': 'mean'})
```

class	fare			survived		
	First	Second	Third	First	Second	Third
female	106.125798	21.970121	16.118810	91.0	70.0	72.0
male	67.226127	19.741782	12.661633	45.0	17.0	47.0

ADDITIONAL PIVOT TABLE OPTIONS

- Example: computation of totals along each grouping – using margins=True → margin labels (row-name and column-name for totals) are ‘All’ by default.
- Instead of ‘All’, margin label can be specified with the margins_name keyword.

```
titanic.pivot_table('survived', index='sex', columns='class', margins = True)
```

class	First	Second	Third	All
sex				
female	0.968085	0.921053	0.500000	0.742038
male	0.368852	0.157407	0.135447	0.188908
All	0.629630	0.472826	0.242363	0.383838